【扔物线 Compose 课程讲义】3.6-重组的性能风险和智能优化、@Stable

扔物线学堂 - 秉心 (1437182)

Recompose 的性能风险

Compose 的自动优化

可靠类型（或称稳定类型）

- 定义
- 实践

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：rengwuxian.com/cp

Recompose 的性能风险

Recompose 过程中，由于会有「被捎带」情况的出现，会重复调用一些没必要重复调用的 Composable 函数，造成性能损耗。

Compose 的自动优化

在 Recompose 过程中，自动跳过参数没变化的 Composable 函数的内部代码。——但参数必须全部都是「可靠类型」。

可靠类型（或称稳定类型）

定义

可以根据 `@Stable` 注解的注释的三个要求来定义：

- 「相等」的恒定性；
- 公开属性的改变自动通知到 Composition；
- 公开属性也全部都是可靠类型。

实践

- 不要轻易重写 `equals()` 函数；
- 用 `by mutableStateOf()` 的方式来代理 `var` 修饰的公开属性（Compose 自动判断类型「可靠」的唯一依据），或者加上 `@Stable` 注解（同时别忘了手动实现「改变自动通知到 Composition 的功能」）；
  - `@Immutable`：含义是「内部不会改变」，即比 `@Stable` 还要稳定，但实际上行为和 `@Stable` 完全一致。
- 对于「公开属性也全部都是可靠类型」，只能靠写代码的时候多注意，但其实一般来说并不需要做任何事。

扔物线学堂 - 秉心 (1437182)